

Mini Project #1

Ryan Webster

2024-02-09

```
# Included libraries
library(dplyr)
library(baseballr)
library(hoopR)
library(lubridate)
library(ProbBayes)
```

Question 1

In order to update the free throw percentage coefficient of the true shooting percentage formula, the total number of free throws attempted should first be found.

Play ID	Type
97	“And One”
98	Free Throws 1 of 2
99	Free Throws 2 of 2
100	Free Throws 1 of 3
101	Free Throws 2 of 3
102	Free Throws 3 of 3
103	Technical
104	Flagrant 1 of 2
105	Flagrant 2 of 2
106	Flagrant 1 of 1
107	Clear Path 1 of 2
108	Clear Path 2 of 2
157	Flagrant 1 of 3
165	Flagrant 2 of 3
166	Flagrant 3 of 3

```
nba_pbp <- hoopR::load_nba_pbp(2016:2021)

ft_nba <- nba_pbp %>%
  # Filters pbp to only include ft
  filter(type_id %in% c(97:108, 157, 165, 166))
total_fts <- nrow(ft_nba) # Count total number of fts
```

This was done by identifying each type of free throw labeled in the NBA play-by-play data from 2016 to 2021 and filtering the data set into a data frame consisting of just free throw attempts. Using this, the total number of free throws shot was found.

Then, the total number of free throw *trips* was found by filtering the play-by-play data again, but this time only counting one of the free throws for each free throw trip to find the total number of trips. (1 of 2, 1 of 3). Note that this calculation omits “and-one” free throws as their made shot attempt has already counted in the calculation and flagrant/technical/clear path fouls as the ft shooting team gets the ball back.

```
# Filters pbp to only include ft trips that end

# possession (so no and one's, technicals, flagrant)
ft_trips <- nba_pbp %>%
  filter(type_id %in% c(98, 100))
total_ft_trips <- nrow(ft_trips) # Counts ft trips
result <- (total_ft_trips / total_fts)
```

It was found that the new coefficient for the free throw attempts is 0.425.

Then, a t-test was conducted to determine if this new coefficient had a statistically significant difference compared to the previous 0.44. First, player box score data was loaded in from 2016-2021. Then, using their total points, field goal/free throw attempts, two columns were created, true shooting percentage using 0.44 and 0.425. A t-test was then conducted to compare the means of these two different true shooting percentages.

```
# Load player boxscore of every game from 2016 to 2021
player_stats <- hoopR::load_nba_player_box(year = 2016:2021)

# groups box scores by players and sums up their

# total points, fga, and fta
season_stats <- player_stats %>%
  group_by(athlete_display_name) %>%
  summarise(
    total_points = sum(points, na.rm = TRUE),
    total_fga = sum(field_goals_attempted, na.rm = TRUE),
    total_fta = sum(free_throws_attempted, na.rm = TRUE))

# Creates a true shooting percentage variable for
# the whole 5 years using both coeff
season_stats$tsa_0.44 <- (season_stats$total_points) /
  (2*(season_stats$total_fga + 0.44*season_stats$total_fta))

season_stats$tsa_0.425 <- (season_stats$total_points) /
  (2*(season_stats$total_fga + result*season_stats$total_fta))

t_test <- t.test(season_stats$tsa_0.44, season_stats$tsa_0.425)
```

Although there isn’t enough evidence to reject the null hypothesis ($p = 0.818$) that the means were even, a slight decrease in the coefficient could result in lower projected number of possessions.

Question 2

For this question, each qualified pitcher (162 innings pitched) and batter (502 PA) per FanGraphs was found shown in the code below, their player id was then used later to extract their boxscores. (2016 was only 60 innings pitched, 185 PA due to COVID).

```

#Finds each pitchers with at least the minimum requirement

#to be a qualified pitcher for each season, extract only their

#name and id.
pitchers_2016 <- fg_pitcher_leaders(startseason = 2016, endseason = 2016,
                                   qual = 162) %>%
  select("PlayerName", "playerid")

pitchers_2017 <- fg_pitcher_leaders(startseason = 2017, endseason = 2017,
                                   qual = 162) %>%
  select("PlayerName", "playerid")

pitchers_2018 <- fg_pitcher_leaders(startseason = 2018, endseason = 2018,
                                   qual = 162) %>%
  select("PlayerName", "playerid")

pitchers_2019 <- fg_pitcher_leaders(startseason = 2019, endseason = 2019,
                                   qual = 162) %>%
  select("PlayerName", "playerid")

pitchers_2020 <- fg_pitcher_leaders(startseason = 2020, endseason = 2020,
                                   qual = 60) %>%
  select("PlayerName", "playerid")

pitchers_2021 <- fg_pitcher_leaders(startseason = 2021, endseason = 2021,
                                   qual = 162) %>%
  select("PlayerName", "playerid")

# Bind together
pitchers <- rbind(pitchers_2016, pitchers_2017, pitchers_2018, pitchers_2019,
                 pitchers_2020, pitchers_2021)

# Eliminate duplicates
unique_pitchers <- pitchers %>%
  distinct(playerid, .keep_all = TRUE)

# Initialize data frame which will keep track of cumulative stats
pitchers_cumulative <- data.frame(PlayerName = character(),
                                  cum_ip = numeric(), WHIP = numeric())

years <- c(2016, 2017, 2018, 2019, 2020, 2021)

```

We then loop through each pitcher and check each year from 2016-2021 and record some necessary statistics to calculate OBS. For players that didn't play in a specific year, that would be skipped. Do note the following code can take ten minutes to run, varying one's hardware.

```

# Loops through each pitcher and extracts their box score for each game.
# Cumulates the innings pitched, hits, and walks and calculates WHIP
for (i in 1:nrow(unique_pitchers)) {
  for( r in 1:length(years)) {
    data <- fg_pitcher_game_logs(unique_pitchers[i, 2], years[r])
    if (nrow(data) == 0) { next }
  }
}

```

```

data <- data %>%
  select(PLAYERNAME, Date, BB, IBB, H, IP) %>%
  mutate(cum_ip = cumsum(IP)) %>%
  mutate(cum_h = cumsum(H)) %>%
  mutate(cum_bb = cumsum(BB)) %>%
  mutate(cum_ibt = cumsum(ibt)) %>%
  mutate(WHIP = (cum_h + cum_bb + cum_ibt) / cum_ip) %>%
  mutate(date = rev(Date)) %>%
  select(PLAYERNAME, date, WHIP, cum_ip)
# Adds onto data frame which will contain every pitcher/year with his stats
pitchers_cumulative <- bind_rows(pitchers_cumulative, data)
}
}

```

Like pitching, the same was done to batting.

```

# Same idea as pitchers, but for batters.
# Minimum 502 PA for qualified hitter
hitters_2016 <- fg_batter_leaders(startseason = 2016, endseason = 2016,
                                qual = 502) %>%
  select("PLAYERNAME", "playerid")
hitters_2017 <- fg_batter_leaders(startseason = 2017, endseason = 2017,
                                qual = 502) %>%
  select("PLAYERNAME", "playerid")
hitters_2018 <- fg_batter_leaders(startseason = 2018, endseason = 2018,
                                qual = 502) %>%
  select("PLAYERNAME", "playerid")
hitters_2019 <- fg_batter_leaders(startseason = 2019, endseason = 2019,
                                qual = 502) %>%
  select("PLAYERNAME", "playerid")
hitters_2020 <- fg_batter_leaders(startseason = 2020, endseason = 2020,
                                qual = 185) %>%
  select("PLAYERNAME", "playerid")
hitters_2021 <- fg_batter_leaders(startseason = 2021, endseason = 2021,
                                qual = 502) %>%
  select("PLAYERNAME", "playerid")

# Bind together
hitters <- rbind(hitters_2016, hitters_2017, hitters_2018, hitters_2019,
                hitters_2020, hitters_2021)

# Eliminate duplicates
unique_hitters <- hitters %>%
  distinct(playerid, .keep_all = TRUE)

# Initialize data frame which will keep track of cumulative stats
hitters_cumulative <- data.frame(PLAYERNAME = character(),
                                cum_ab = numeric(), ops = numeric())

```

Again, a nested loop was constructed to go through every pitcher and year to grab the necessary statistics.

```

# Loops through each pitcher and extracts their box score for each game.
# Cumulates the at bats, hits, type of hit, walks, etc. to calculate obs
for (i in 1:nrow(unique_hitters)) {
  for( r in 1:length(years)) {
    data <- fg_batter_game_logs(unique_hitters[i, 2], years[r])
    if (nrow(data) == 0) { next }
    data <- data %>%
      select(PLAYERNAME, Date, AB, H, `1B`, `2B`, `3B`,
             HR, BB, IBB, HBP, SF) %>%
      mutate(cum_ab = cumsum(AB)) %>%
      mutate(cum_h = cumsum(H)) %>%
      mutate(cum_1b = cumsum(`1B`)) %>%
      mutate(cum_2b = cumsum(`2B`)) %>%
      mutate(cum_3b = cumsum(`3B`)) %>%
      mutate(cum_hr = cumsum(HR)) %>%
      mutate(cum_bb = cumsum(BB)) %>%
      mutate(cum_ibt = cumsum(ibt)) %>%
      mutate(cum_hbp = cumsum(HBP)) %>%
      mutate(cum_sf = cumsum(SF)) %>%
      # total bases calculation
      mutate(cum_tb = cum_h + (2 * cum_2b) + (3 * cum_3b) + (4 * cum_hr)) %>%
      # slugging calculation
      mutate(slg = cum_h / cum_ab) %>%
      # obp calculation
      mutate(obp = (cum_h + cum_bb + cum_ibt + cum_hbp) /
                 (cum_ab + cum_bb + cum_ibt + cum_hbp + cum_sf)) %>%
      # ops calculation
      mutate(ops = slg + obp) %>%
      mutate(date = rev(Date)) %>%
      select(PLAYERNAME, date, cum_ab, ops)
    # Adds onto data frame which will contain every batter/year with his stats
    hitters_cumulative <- bind_rows(hitters_cumulative, data)
  }
}

```

Part A

The data was filtered to use only the values post 2019 season and up to May for seasons 2019-2022. An estimate prior was created by using the ‘average’ ops of a ball player and an estimated standard deviation. This was done using the ProbBayes package and using the normal_update() function which finds the normal posterior given normal data and prior. This will be used throughout this question.

It was found that, by using the data post 2019, the predicted OPS of that span was 0.633. For pitchers, the expected WHIP was 1.23. Again for pitchers, it was found that the mean WHIP for post May was 1.39, 0.16 over the predicted value. For batters, it was found that the mean OPS for post May was 0.59, 0.04 off the predicted OPS. Overall, the predictions were able to predict their respected metrics within less than one standard deviation, so it’s fair to say it did a good job forecasting.

```

# Filters cumulative data to only contain post 2019 and before the end of May
hitter_predictive <- hitters_cumulative %>%
  filter(year(date) >= 2019) %>%
  filter(month(date) < 06) %>%
  filter(!ops == 0)

```

```

# Used to measure prediction with end of season results
end_season_ops <- hitters_cummulative %>%
  filter(year(date) >= 2019) %>%
  filter(month(date) >= 06) %>%
  filter(!ops == 0)

# "Average" baseball player's OPS and estimated standard deviation for the prior
mu_ops = 0.65
sd_ops = 0.1

# Data used
data_mu_ops = mean(hitter_predictive$ops, na.rm = TRUE)
data_sd_ops = sd(hitter_predictive$ops, na.rm = TRUE)

# Use ProbBayes library to produce posterior
prior_ops <- c(mu_ops, sd_ops)
data_ops <- c(data_mu, data_sd)
result_ops <- normal_update(prior_ops, data_ops, teach = TRUE)
#posterior = 0.633

end_mean_obp <- mean(end_season$ops) #0.59

# Filters cumulative data to only contain
# post 2019 and before the end of May
pitcher_predictive <- pitchers_cummulative %>%
  filter(year(date) >= 2019) %>%
  filter(month(date) < 06)

# Used to measure prediction with end of season results
end_season_whip <- pitchers_cummulative %>%
  filter(year(date) >= 2019) %>%
  filter(month(date) >= 06)

# Remove infinite WHIP's
pitcher_predictive <- pitcher_predictive[!is.infinite(pitcher_predictive$WHIP), ]
end_season_whip <- end_season_whip[!is.infinite(end_season_whip$WHIP), ]

# "Average" baseball player's whip and estimated
# standard deviation for the prior
mu_whip = 1.2
sd_whip = 0.6

# Data used
data_mu_whip = mean(pitcher_predictive$WHIP)
data_sd_whip = sd(pitcher_predictive$WHIP)

# Use ProbBayes library to produce posterior
prior_whip <- c(mu_whip, sd_whip)
data_whip <- c(data_mu_whip, data_sd_whip)
result_whip <- normal_update(prior_whip, data_whip, teach = TRUE)
# posterior = 1.23 WHIP

end_mean_whip <- mean(end_season_whip$WHIP, na.rm = TRUE) #1.39

```

Part B

Now, we will take individual player data from 2016-2018 to predict how each player will perform in the second half of the 2019 season. Each of the batters are then assigned their respected predictions as to what their OPS should be given the prior. Then, the predicted OPS is compared to each player's OPS at the end of the 2019 season. It's shown that the mean difference is 0.05 OPS.

```
# Creates empty data frame
prior_player <- data.frame(player = unique_hitters$PlayerName)

# Takes data from before 2019 to use as prior,
# uses mean/sd obs for the bayes
prior_obs <- hitters_cummulative %>%
  filter(year(date) < 2019) %>%
  filter(!ops == 0) %>%
  group_by(PlayerName) %>%
  summarise(mean = mean(ops),
            sd = sd(ops))

# Applies the prior and data through the normal bayes model.
# each result is then assigned to the respected players row
for (i in 1:nrow(prior_obs)) {
  prior <- c(prior_obs$mean[i], prior_obs$sd[i])
  result <- normal_update(prior, data_ops, teach = TRUE)
  prior_obs$prediction[i] <- result[3, 2]
}

# finds each players obs at the end of the 2019 season
end_2019_season_batter <- hitters_cummulative %>%
  filter(year(date) == 2019) %>%
  filter(month(date) >= 6) %>%
  group_by(PlayerName) %>%
  slice(n())

# Merge to put end of season OPS onto the dataset with the predictions
prior_obs <- merge(prior_obs, end_2019_season_batter,
                  by = "PlayerName", all.x = T) %>%
  select(PlayerName, mean, sd, prediction, ops)

# absolute difference is then calculated
prior_obs$difference <- abs(prior_obs$prediction - prior_obs$ops)

mean(prior_obs$difference, na.rm = TRUE)
```

For the pitching side, the same idea was applied. It was found that the mean absolute difference of each pitcher's predicted WHIP was 0.2 off. It's worth noting that this result wasn't more accurate than the previous part, this could be due to many factors that affect pitchers daily such as health, ballpark, weather, competition, etc.

```
prior_pitchers <- data.frame(player = unique_pitchers$PlayerName)

# Takes data from before 2019 to use as prior,
# uses mean/sd OPS for the bayes
prior_whip <- pitchers_cummulative %>%
```

```

filter(year(date) < 2019) %>%
group_by(PlayerName) %>%
summarise(mean = mean(WHIP),
          sd = sd(WHIP))

prior_whip <- prior_whip[!is.infinite(prior_whip$mean), ]
prior_whip <- na.omit(prior_whip)

# Applies the prior and data through the normal bayes model.
# each result is then assigned to the respected players row
for (i in 1:nrow(prior_whip)) {
  prior <- c(prior_whip$mean[i], prior_whip$sd[i])
  result <- normal_update(prior, data_whip, teach = TRUE)
  prior_whip$prediction[i] <- result[3, 2]
}

# finds each players obs at the end of the 2019 season
end_2019_season_pitchers <- pitchers_cumulative %>%
  filter(year(date) == 2019) %>%
  filter(month(date) >= 6) %>%
  group_by(PlayerName) %>%
  slice(n())

# Merge to put end of season obs onto the dataset with the predictions
prior_whip <- merge(prior_whip, end_2019_season_pitchers,
                    by = "PlayerName", all.x = T) %>%
  select(PlayerName, mean, sd, prediction, WHIP.y)

# absolute difference is then calculated
prior_whip$difference <- abs(prior_whip$prediction - prior_whip$WHIP.y)

mean(prior_whip$difference, na.rm = TRUE)

```

Part C

For this part, we will compare the prediction with the rate of each player's OPS going into May. It is shown that the mean of the absolute difference is 0.05, about the same as using the previous strategy.

```

# Creates dataset which only has stats in the end of May
# and finds their mean and sd.
end_may_obs <- hitters_cumulative %>%
  filter(year(date) == 2019) %>%
  filter(month(date) == 5) %>%
  group_by(PlayerName) %>%
  summarise(mean = mean(ops),
            sd = sd(ops))

# Uses the normal bayes posterior to create predictions
end_may_obs$prediction <- NA
for (i in 1:nrow(end_may_obs)) {
  prior <- c(end_may_obs$mean[i], end_may_obs$sd[i])
  result <- normal_update(prior, data_ops, teach = TRUE)
}

```

```

    end_may_obs$prediction[i] <- result[3, 2]
  }

  # Merge predictions with batter obs
  end_may_obs <- merge(end_may_obs, end_2019_season_batter,
    by = "PlayerName", all.x = T) %>%
    select(PlayerName, mean, sd, prediction, ops.y)

  # Finds difference
  end_may_obs$difference <- abs(end_may_obs$prediction - end_may_obs$ops.y)

  mean(end_may_obs$difference, na.rm = TRUE)

```

The same idea is then applied to pitching. It was shown that the difference was about 0.2, around the same as the previous technique.

```

# Creates dataset which only has stats in the end of May
# and finds their mean and sd.
end_may_whip <- pitchers_cumulative %>%
  filter(year(date) == 2019) %>%
  filter(month(date) == 5) %>%
  group_by(PlayerName) %>%
  summarise(mean = mean(WHIP),
    sd = sd(WHIP))

# Uses the normal bayes posterior to create predictions
end_may_whip$prediction <- NA
for (i in 1:nrow(end_may_whip)) {
  prior <- c(end_may_whip$mean[i], end_may_whip$sd[i])
  result <- normal_update(prior, data_whip, teach = TRUE)
  end_may_whip$prediction[i] <- result[3, 2]
}

# Merge predictions with pitcher whip
end_may_whip <- merge(end_may_whip, end_2019_season_pitchers,
  by = "PlayerName", all.x = T) %>%
  select(PlayerName, mean, sd, prediction, WHIP.y)

# Finds difference
end_may_whip$difference <- abs(end_may_whip$prediction - end_may_whip$WHIP.y)

mean(end_may_whip$difference, na.rm = TRUE)

```

Overall, it was shown that the techniques in part b were better than in part a, only in predicting OPS, however. Surprisingly, the technique of looking at the rate midway through the season was just as accurate. Perhaps this is due to the fact that already half way through the season, a player's OPS/WHIP has already regressed towards the true mean. We can learn that that typically, the more data used to create a prediction into the future, the better. To improve the data and create better predictions, one could use factors like player health, ballpark factors, weather, performance of his team, etc..